

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Agile Sprint Planning & User Story Workshop
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	V. Karthik Harish	Reg. No.:	192511328
Date:	08/10/2026	Duration:	60 minutes

Code of Conduct

I V. Karthik Harish (192511328) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **V. Karthik Harish**

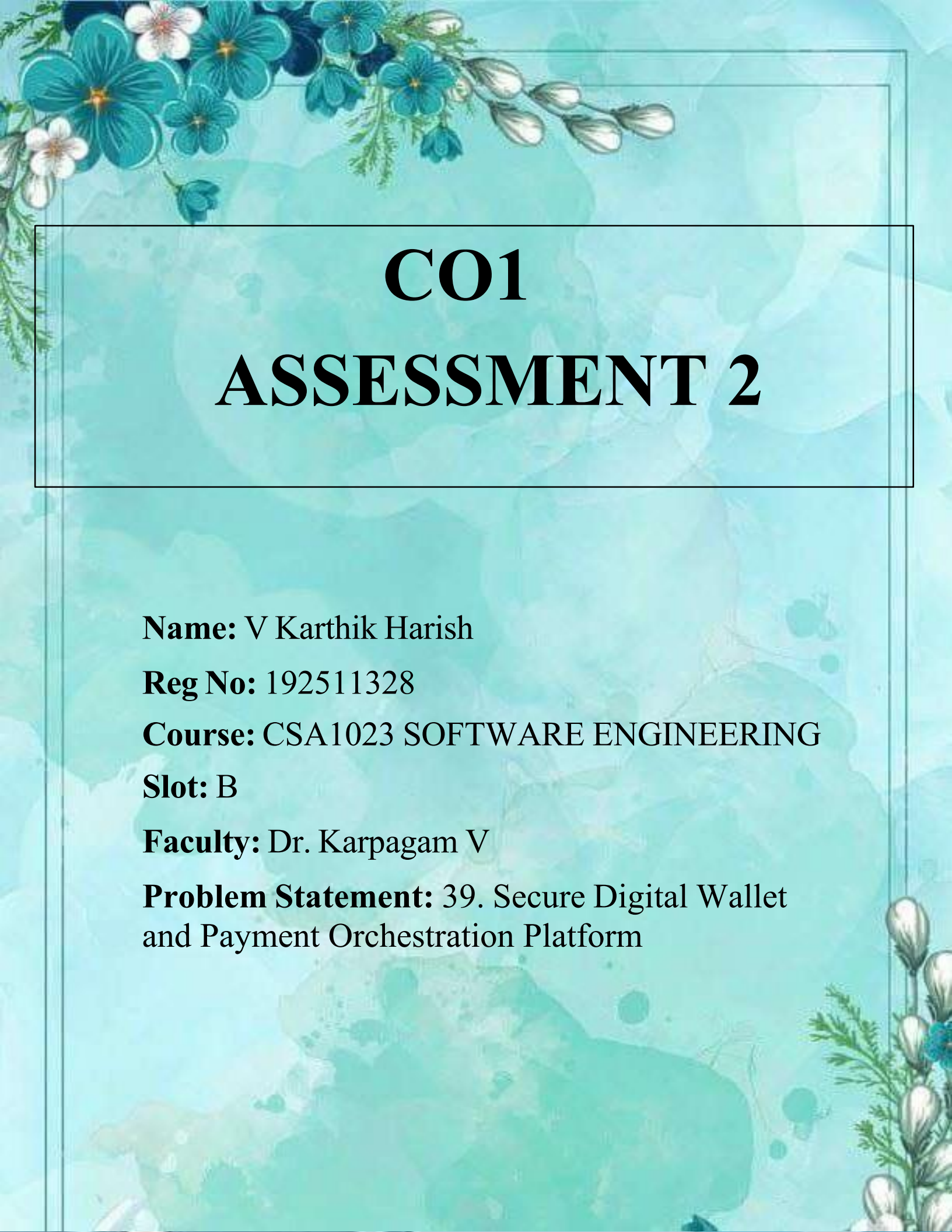
Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

- 1. Identify the Project Vision and Stakeholders**
 - Define the project's vision, objectives, and identify all key stakeholders involved.
- 2. Create Epics and User Stories**
 - Break down the requirements into Epics and formulate well-structured User Stories using the format:
As a <user>, I want <feature>, so that <benefit>.
- 3. Define Acceptance Criteria**
 - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
- 4. Prioritize the Product Backlog**
 - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
- 5. Plan the Sprint Backlog**
 - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
- 6. Estimate Effort Using Story Points**
 - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
- 7. Present the Sprint Goal and Expected Deliverables**
 - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.



C01

ASSESSMENT 2

Name: V Karthik Harish

Reg No: 192511328

Course: CSA1023 SOFTWARE ENGINEERING

Slot: B

Faculty: Dr. Karpagam V

Problem Statement: 39. Secure Digital Wallet and Payment Orchestration Platform

Workshop Overview

Item	Details
Product	Secure Digital Wallet and Payment Orchestration Platform (working name: SecurePay Orchestrator)
Framework	Scrum, supported by DevOps (CI/CD) practices
Sprint length	2 weeks (10 working days)
Scrum team (assumed)	Product Owner, Scrum Master and 5 developers: 2 backend, 1 frontend, 1 QA/test, 1 DevOps
Estimation technique	Story points on the Fibonacci scale (1, 2, 3, 5, 8, 13) using Planning Poker
Product backlog	10 epics, 31 user stories, 146 story points
Sprint 1 commitment	8 user stories, 29 story points, 35 development tasks plus 4 team tasks (204 planned hours)

1. Project Vision and Stakeholders

1.1 Project Vision

For **customers and merchants** who need to pay and be paid digitally without the fear of failed, duplicate or disputed transactions, **SecurePay Orchestrator** is a **secure digital wallet and payment orchestration platform** that coordinates every payment across multiple services, keeps each transaction in one consistent state, and recovers automatically when a service fails. **Unlike** point-to-point integrations that leave money in limbo and depend on manual reconciliation, our product guarantees once-only processing, a complete audit trail and real-time visibility for everyone involved.

Vision in one line: *Every payment completes exactly once, can always be explained, and never leaves money in limbo.*

1.2 Product Objectives and Success Measures

#	Objective (from the problem statement)	Measure of success
O1	Process and track digital payment transactions	100% of requests receive a unique transaction ID and a queryable status
O2	Maintain consistent transaction states	0 transactions in an illegal state; at least 99.9% of transactions reach a final state within 15 minutes
O3	Prevent duplicate transaction processing	0 duplicate debits when 50 identical requests are sent in

#	Objective (from the problem statement)	Measure of success
		parallel
O4	Integrate multiple payment services securely	A new gateway needs only one new adapter, with no change to orchestration logic; all calls use TLS and signed requests
O5	Provide real-time monitoring and alerts	An alert is raised within 2 minutes of a threshold breach
O6	Implement testing, CI/CD and incident recovery	Every commit passes the automated pipeline; test coverage of at least 80%; failed payments are reversed automatically
O7	Maintain secure and auditable transaction records	Every state change is stored in an append-only audit log; any transaction can be reconstructed from the log

1.3 Scope Boundary

In scope: authentication, wallets, payment orchestration, transaction states, idempotency, gateway integration, recovery and reconciliation, ledger and audit, monitoring, CI/CD and an operations dashboard. **Out of scope:** building a bank or settlement system, lending products and native mobile apps; the platform exposes APIs that any client can use.

1.4 Key Stakeholders

Stakeholder	Role and responsibilities	What they need from the product	Influence / Interest	How they are engaged in Scrum
Product Owner (business sponsor)	Owns the vision and product backlog; orders priorities; accepts or rejects increments	Maximum business value per sprint	High / High	Sprint planning, refinement, sprint review
Scrum Master	Coaches the team on Scrum; removes impediments; protects the sprint	A team that is focused and unblocked	Medium / High	Facilitates all events
Development team (backend,	Design, build, test and release the	Clear stories, acceptance criteria	High / High	Daily stand-up; estimation; sprint

Stakeholder	Role and responsibilities	What they need from the product	Influence / Interest	How they are engaged in Scrum
frontend, QA, DevOps)	increment	and a stable sprint		events
Customers (wallet users)	Register, top up, pay, raise disputes	Fast, safe payments; never charged twice; clear status	Medium / High	Feedback in sprint review; usability testing
Merchants	Receive payments, request refunds, reconcile settlements	Reliable payment status; correct settlements	High / High	Representative joins sprint review
Payment gateways, banks and card networks	Authorise and settle money; send callbacks	Valid, de-duplicated and well-formed requests	High / Medium	Integration contract and sandbox testing
Operations / SRE team	Monitor, respond to incidents, run recovery	Dashboards, alerts, runbooks	Medium / High	Review of monitoring stories; incident feedback
Security and compliance officers	Define policy; audit; ensure PCI DSS-style and data-protection compliance	Encryption, access control, audit evidence	High / Medium	Security acceptance criteria; Definition of Done review
Finance and dispute-resolution team	Reconcile ledger with gateway reports; resolve disputes and chargebacks	Balanced ledger; automatic mismatch reports	Medium / High	Review of ledger and reconciliation stories
Customer support agents	Answer customer queries	Searchable transaction timeline	Low / High	Demo of the operations dashboard
Regulators and external auditors	Verify compliance and traceability	Complete, tamper-evident records	High / Low	Kept informed through compliance reports

2. Epics and User Stories

2.1 Epics

The product requirements are grouped into ten epics. Each epic maps to one or more objectives of the problem statement.

Epic	Name	Goal	Stories	Points	Objectives
E1	Identity and Access Management	Only verified, authorised people can use the platform	US-01, 02, 03, 31	16	O1, O7
E2	Wallet Management	Customers hold and move funds in a wallet	US-04, 05, 06	10	O1
E3	Payment Orchestration and Transaction State	Every payment follows one controlled life-cycle	US-07, 08, 09	15	O1, O2
E4	Idempotency and Duplicate Prevention	A request is processed at most once	US-10, 11	7	O3
E5	Gateway Integration	Several payment services are used through one interface	US-12, 13, 14, 15	24	O4
E6	Recovery and Reconciliation	Failures are corrected automatically	US-16, 17, 18, 19	21	O2, O6
E7	Ledger, Audit and Security	Money movements are secure and provable	US-20, 21, 22	18	O4, O7
E8	Monitoring and Alerting	Problems are seen and acted on in real time	US-23, 24, 25	11	O5
E9	Quality Engineering and CI/CD	Every change is tested and safely released	US-26, 27	8	O6

Epic	Name	Goal	Stories	Points	Objectives
E10	Notifications, Refunds and Operations Support	Users and staff can follow and correct transactions	US-28, 29, 30	16	O1, O7

2.2 User Stories

All stories use the format **As a <user>, I want <feature>, so that <benefit>**. The last column gives the key acceptance criterion; the full Given–When–Then criteria for the Sprint 1 stories are in Section 3.

Epic E1 – Identity and Access Management

ID	User story	Key acceptance criterion
US-01	As a customer, I want to register and verify my account, so that I can use the wallet securely.	Account becomes ACTIVE only after OTP verification; duplicate email is rejected
US-02	As a registered user, I want to log in securely, so that only I can access my wallet.	Valid login returns a token that expires in 15 minutes; 5 failed attempts lock the account for 15 minutes
US-03	As a customer, I want to confirm high-value payments with a one-time password, so that a stolen session cannot move large sums.	Payments above the set limit need a valid OTP; an OTP expires after 5 minutes
US-31	As an administrator, I want role-based access control, so that customers, merchants and support staff see only what their role allows.	A request outside the role returns 403 and is logged

Epic E2 – Wallet Management

ID	User story	Key acceptance criterion
US-04	As a new customer, I want a wallet created automatically when my account is verified, so that I can start transacting immediately.	Exactly one wallet with zero balance exists per verified user
US-05	As a customer, I want to view my balance and transaction history, so that I can track my money.	Balance equals the ledger total; history is paginated and loads in under 2 seconds

ID	User story	Key acceptance criterion
US-06	As a customer, I want to top up my wallet, so that I can pay without entering card details every time.	A successful top-up credits the wallet exactly once; a failed top-up leaves the balance unchanged

Epic E3 – Payment Orchestration and Transaction State

ID	User story	Key acceptance criterion
US-07	As a customer, I want to pay a merchant from my wallet, so that I can buy goods and services quickly.	A payment either completes (customer debited, merchant credited) or fails with no money lost
US-08	As a platform operator, I want every transaction governed by a state machine, so that its status is always valid and consistent.	Only legal transitions are accepted; an illegal transition is rejected and logged
US-09	As a merchant, I want to query the status of a payment, so that I know whether to release goods or services.	Status API returns the current state and time in under 500 ms for 95% of calls

Epic E4 – Idempotency and Duplicate Prevention

ID	User story	Key acceptance criterion
US-10	As a customer, I want a retried request to be processed only once, so that I am never charged twice.	The same key within 24 hours returns the original result and causes one debit only
US-11	As a platform operator, I want a reused key with a different request body rejected, so that client bugs cannot corrupt transactions.	The request returns 422 and no state changes

Epic E5 – Gateway Integration

ID	User story	Key acceptance criterion
US-12	As a developer, I want a common gateway interface with a mock gateway, so that I can build and test payment flows without a real provider.	Interface offers charge, status and refund; mock supports success, decline and timeout modes
US-13	As a platform operator, I want to	A sandbox payment completes

ID	User story	Key acceptance criterion
	connect a real payment gateway through the adapter, so that real payments can be processed.	end to end; credentials come from the secrets vault
US-14	As a platform operator, I want gateway webhooks verified before processing, so that forged callbacks cannot change a transaction.	An invalid signature returns 401 with no state change; a duplicate webhook is processed once
US-15	As a merchant, I want payments rerouted to a backup gateway when the primary is down, so that my customers can still pay.	Once the circuit opens, new payments use the backup gateway within 30 seconds

Epic E6 – Recovery and Reconciliation

ID	User story	Key acceptance criterion
US-16	As a customer, I want failed payments reversed automatically, so that my money is never stuck.	After a failure the hold is released within 60 seconds and the ledger balances
US-17	As a platform operator, I want temporary errors retried with back-off and a circuit breaker, so that short outages do not become failed payments.	Up to 3 retries with exponential back-off; the breaker opens after 5 consecutive failures
US-18	As a finance officer, I want daily automatic reconciliation against gateway reports, so that mismatches are found and fixed quickly.	Every mismatch is listed with a reason; mismatches covered by a rule are corrected automatically
US-19	As an operations engineer, I want messages that keep failing moved to a dead-letter queue, so that nothing is lost and I can investigate.	A message moves after 5 failed attempts and an alert is raised

Epic E7 – Ledger, Audit and Security

ID	User story	Key acceptance criterion
US-20	As a finance officer, I want every money movement recorded in a double-entry	Total debits equal total credits for every transaction

ID	User story	Key acceptance criterion
	ledger, so that the books always balance.	
US-21	As an auditor, I want an append-only, tamper-evident audit log, so that I can trust and reconstruct any transaction.	Updates and deletes are blocked; hash-chain verification detects any tampering
US-22	As a security officer, I want sensitive data encrypted and secrets kept in a vault, so that a breach does not expose card data or credentials.	No raw card number or CVV is stored; AES-256 at rest and TLS 1.2 or higher in transit

Epic E8 – Monitoring and Alerting

ID	User story	Key acceptance criterion
US-23	As an operations engineer, I want a real-time dashboard of success rate, latency and stuck transactions, so that I can see platform health at a glance.	Dashboard metrics refresh within 30 seconds
US-24	As an operations engineer, I want alerts when thresholds are breached, so that I can react before customers complain.	An alert is delivered within 2 minutes of the breach
US-25	As a developer, I want structured logs with a correlation ID for each transaction, so that I can trace a payment across services.	Every log line of a transaction carries the same ID; no sensitive data appears in logs

Epic E9 – Quality Engineering and CI/CD

ID	User story	Key acceptance criterion
US-26	As a developer, I want every commit built and tested automatically, so that defects are caught before merge.	The pipeline runs on every pull request in under 10 minutes; merging is blocked when it fails
US-27	As a DevOps engineer, I want automated deployment to staging and production with rollback, so that releases are fast and safe.	No manual step except approval; rollback completes in under 5 minutes

Epic E10 – Notifications, Refunds and Operations Support

ID	User story	Key acceptance criterion
US-28	As a customer, I want an SMS or email about each payment outcome, so that I know immediately what happened.	Notification is sent within 30 seconds of the final state
US-29	As a support agent, I want to search a transaction and see its full timeline, so that I can resolve disputes quickly.	Search by ID, user or date returns in under 3 seconds; timeline lists every state change with time
US-30	As a merchant, I want to refund a completed payment, so that I can handle returns and cancellations.	Refund creates reversing ledger entries; refund cannot exceed the original amount

Quality check: every story was written to satisfy the INVEST checklist (Independent, Negotiable, Valuable, Estimable, Small, Testable). Stories above 8 points are split before entering a sprint, which is why no story exceeds 8 points. A story enters a sprint only when it meets the **Definition of Ready:** written in the standard format, acceptance criteria agreed, estimated, dependencies identified and small enough to finish within one sprint.

3. Acceptance Criteria (Given–When–Then)

Every story selected for Sprint 1 has specific, measurable and testable acceptance criteria. Each scenario below becomes an automated test case and is checked by the QA engineer before the Product Owner accepts the story.

US-01 – Register and verify account

Scenario	Given	When	Then
Successful registration	A visitor with a new email and a password of at least 10 characters including a number and a symbol	They submit the registration form	An account is created with status PENDING, the password is stored only as a salted hash, and a 6-digit OTP is sent
Successful verification	A PENDING account and the correct OTP entered within 5 minutes	The user submits the OTP	The account status changes to ACTIVE
Invalid attempts	An email that is already registered, or an expired or wrong OTP	The user registers or verifies	The request is rejected with a clear message (409 for a duplicate) and no account status

Scenario	Given	When	Then
			changes

US-02 – Secure login

Scenario	Given	When	Then
Valid login	An ACTIVE user with correct credentials	They log in	An access token valid for 15 minutes and a refresh token are returned in under 1 second for 95% of requests
Account lockout	5 consecutive wrong passwords for one account	A sixth attempt is made	The account is locked for 15 minutes, the response is 423 and the event is logged
Invalid token	An expired or tampered token	A protected API is called	The response is 401 and no data is returned

US-04 – Automatic wallet creation

Scenario	Given	When	Then
Wallet created	A user whose account has just been verified	Verification completes	Exactly one wallet with balance 0.00 is created in the same database transaction
Creation failure	Wallet creation fails	Verification is processed	The account stays PENDING and the user can retry
Replayed event	A user who is already ACTIVE	The verification event is replayed	No second wallet is created

US-12 – Gateway interface and mock gateway

Scenario	Given	When	Then
Common contract	The PaymentGateway interface	A developer calls charge, status or refund on the mock	Each call returns the common response format: status, gateway reference and error code
Decline mode	The mock is set to	A charge is made	The response is DECLINED with a

Scenario	Given	When	Then
	decline mode		reason code
Timeout mode	The mock is set to timeout mode	A charge is made	No reply arrives within 5 seconds and the caller receives a timeout error
Contract tests	Any gateway adapter	The contract test suite runs	All methods satisfy the same contract

US-26 – Automated CI pipeline

Scenario	Given	When	Then
Pipeline runs	A developer opens a pull request	The pipeline starts	Build, lint, unit tests and the security scan run automatically and results appear on the pull request within 10 minutes
Quality gate	A failing test or coverage below 70%	The pipeline finishes	Merging is blocked
Vulnerable dependency	A dependency with a high-severity vulnerability	The scan runs	The pipeline fails and names the dependency

US-08 – Core transaction state machine

Scenario	Given	When	Then
Successful path	A transaction in INITIATED	The gateway call starts and then confirms	The state moves INITIATED, PROCESSING, SUCCESS and each change is stored with a timestamp
Failure path	A transaction in PROCESSING	The gateway declines	The state becomes FAILED with the reason stored
Illegal transition	A transaction in SUCCESS	A request tries to move it back to PROCESSING	The request is rejected, the state is unchanged and the attempt is logged
Concurrent updates	Two simultaneous updates to the same	Both try to commit	Only one succeeds (version check); the

Scenario	Given	When	Then
	transaction		other is rejected or retried

US-06 – Wallet top-up

Scenario	Given	When	Then
Successful top-up	A logged-in customer with an ACTIVE wallet	They top up 500.00 and the gateway approves	The balance increases by exactly 500.00 once and the transaction ends in SUCCESS
Declined top-up	The gateway declines the charge	Processing finishes	The balance is unchanged, the transaction is FAILED and the customer sees the reason
Gateway timeout	The gateway gives no reply within 5 seconds	The timeout is reached	The transaction is FAILED with reason TIMEOUT and the balance is unchanged (the UNKNOWN state and reconciliation come later with US-16 and US-18)
Invalid amount	An amount of zero, a negative amount or an amount above the limit	The customer submits the top-up	The request returns 400 and no transaction is created

US-10 – Idempotent requests

Scenario	Given	When	Then
First request	A request with a new Idempotency-Key	It is processed	The key, a hash of the request and the response are stored and one transaction is created
Replayed request	The same key and the same body within 24 hours	The request is sent again	The original response is returned, marked as a replay, and no new transaction or credit is made

Scenario	Given	When	Then
Parallel duplicates	50 identical requests with the same key sent at the same time	All arrive together	Exactly one transaction exists and the wallet is credited once
Key expiry	A key older than 24 hours	The clean-up job runs	The key is removed and the job result is logged

4. Product Backlog Prioritisation

4.1 Prioritisation Approach

The Product Owner and team ordered the backlog using four criteria, applied in this order:

1. **MoSCoW category** (Must, Should, Could): Must-have items are required for a safe, working payment platform.
2. **Dependencies**: a story cannot be built before the stories it depends on.
3. **Business value (1–10)**: scored by the Product Owner using customer need, risk reduction and compliance impact.
4. **Risk first**: the riskiest technical items (state machine, idempotency, real gateway) are scheduled early so that problems surface while change is still cheap.

Won't have in this release: AI-based fraud detection, multi-currency and cross-border payments, and a self-service dispute portal (recorded as future enhancements).

4.2 Prioritised Product Backlog

Rank	ID	Story	MoSCoW	Value	Depends on	Points	Sprint
1	US-26	Automated CI pipeline	Must	7	None	3	1
2	US-01	Register and verify account	Must	9	None	3	1
3	US-02	Secure login	Must	9	US-01	3	1
4	US-04	Automatic wallet creation	Must	8	US-01	2	1
5	US-12	Gateway interface and mock gateway	Must	9	None	3	1

Rank	ID	Story	MoSCoW	Value	Depends on	Points	Sprint
6	US-08	Core transaction state machine	Must	10	None	5	1
7	US-10	Idempotent requests	Must	10	US-08	5	1
8	US-06	Wallet top-up	Must	8	US-04, 08, 10, 12	5	1
9	US-31	Role-based access control	Must	8	US-02	5	2
10	US-07	Pay a merchant from wallet	Must	10	US-06, 08, 10	8	2
11	US-03	OTP for high-value payments	Should	8	US-02	5	2
12	US-11	Reject reused key with different body	Should	7	US-10	2	2
13	US-09	Payment status query	Must	8	US-08	2	2
14	US-05	Balance and history	Should	6	US-06	3	2
15	US-25	Structured logs with correlation ID	Must	7	None	3	2
16	US-13	Real gateway integration	Must	9	US-12	8	3

Rank	ID	Story	MoSCoW	Value	Depends on	Points	Sprint
17	US-20	Double-entry ledger	Must	10	US-07	8	3
18	US-14	Webhook verification	Must	8	US-13	5	3
19	US-22	Encryption and secrets vault	Must	9	None	5	3
20	US-16	Automatic reversal	Must	9	US-07, 20	5	4
21	US-17	Retry and circuit breaker	Must	8	US-13	5	4
22	US-18	Daily reconciliation	Must	8	US-13, 20	8	4
23	US-19	Dead-letter queue	Should	6	US-17	3	4
24	US-21	Append-only audit log	Must	9	US-08	5	4
25	US-15	Failover to backup gateway	Should	7	US-17	8	5
26	US-23	Real-time metrics dashboard	Should	7	US-25	5	5
27	US-24	Threshold alerts	Should	7	US-23	3	5
28	US-30	Refunds	Should	6	US-07, 20	5	5
29	US-27	Automated CD and rollback	Should	7	US-26	5	5

Rank	ID	Story	MoSCoW	Value	Depends on	Points	Sprint
30	US-28	Payment notifications	Could	5	US-07	3	5
31	US-29	Operations dashboard	Could	6	US-21, 23	8	6

4.3 Indicative Release Roadmap

Sprint	Theme	Points
1	Foundation: a thin end-to-end flow (register, log in, top up with a mock gateway) with CI	29
2	Payments and access control: merchant payments, RBAC, OTP, status API, logging	28
3	Real integration, ledger and security: real gateway, webhooks, ledger, encryption	26
4	Resilience and audit: reversal, retries, reconciliation, dead-letter queue, audit log	26
5	Failover, observability and release automation: backup gateway, metrics, alerts, refunds, CD, notifications	29
6	Operations support and hardening: operations dashboard, load and security testing, runbooks	8 plus hardening work

Why this order: Sprint 1 builds a “walking skeleton”, the thinnest working path through the whole system, so that integration risk is found in the first two weeks. Security and money-correctness items (ledger, encryption, audit) are Must-haves and are scheduled before convenience features such as notifications and the operations dashboard. The Product Owner re-ranks the backlog at every refinement session as feedback arrives.

5. Sprint 1 Backlog Planning

5.1 Stories Selected for Sprint 1

ID	Story	Points	Why it belongs in Sprint 1
US-26	Automated CI pipeline	3	Every later story depends on automated testing and quality gates
US-01	Register and verify account	3	Entry point for every user journey
US-02	Secure login	3	Needed to protect every API
US-04	Automatic wallet creation	2	A wallet is needed before any money can move
US-12	Gateway interface and mock gateway	3	Lets the team build and test payment flows without a real provider
US-08	Core transaction state machine	5	Highest-risk, highest-value domain logic; everything else builds on it
US-10	Idempotent requests	5	Core promise of the product: never process a request twice
US-06	Wallet top-up	5	Brings all the pieces together in one demonstrable end-to-end flow
	Total	29	

5.2 Task Breakdown

Roles: BE1 and BE2 (backend developers), FE (frontend developer), QA (test engineer), DevOps (DevOps engineer). Estimates are in hours.

US-26 – Automated CI pipeline (3 points, 16 hours)

Task	Description	Role	Hours
26.1	Configure repository,	DevOps	3

Task	Description	Role	Hours
	branch protection, branching strategy and pull request template		
26.2	Create CI workflow: build, lint and unit tests on every pull request	DevOps	6
26.3	Add dependency and static security scan step	DevOps	4
26.4	Add coverage report and a 70% quality gate	QA	3

US-01 – Register and verify account (3 points, 24 hours)

Task	Description	Role	Hours
01.1	Design user table and registration API contract	BE1	3
01.2	Implement registration endpoint with validation and password hashing	BE1	6
01.3	Implement OTP verification with a mock provider	BE1	5
01.4	Registration and verification screens	FE	5
01.5	Unit and integration tests for all three scenarios	QA	5

US-02 – Secure login (3 points, 21 hours)

Task	Description	Role	Hours
02.1	Login endpoint with access and refresh tokens	BE1	6
02.2	Account lockout after 5 failed attempts	BE1	3
02.3	Authentication middleware for	BE2	3

Task	Description	Role	Hours
	protected routes		
02.4	Login screen and token handling	FE	5
02.5	Tests including expired and tampered tokens	QA	4

US-04 – Automatic wallet creation (2 points, 9 hours)

Task	Description	Role	Hours
04.1	Wallet table and wallet service	BE2	4
04.2	Create wallet in the same database transaction as verification	BE2	3
04.3	Tests for success, failure and replay	QA	2

US-12 – Gateway interface and mock gateway (3 points, 13 hours)

Task	Description	Role	Hours
12.1	Define the PaymentGateway interface (charge, status, refund)	BE2	3
12.2	Build mock gateway with success, decline and timeout modes	BE2	6
12.3	Write contract tests that any adapter must pass	QA	4

US-08 – Core transaction state machine (5 points, 24 hours)

Task	Description	Role	Hours
08.1	Model states and the table of legal transitions	BE1	3
08.2	Transaction and state-history tables with version column	BE2	5
08.3	Transition service that	BE1	8

Task	Description	Role	Hours
	rejects illegal moves		
08.4	Parameterised tests for every legal and illegal transition	QA	6
08.5	Document the state diagram in the repository	BE1	2

US-10 – Idempotent requests (5 points, 24 hours)

Task	Description	Role	Hours
10.1	Idempotency table with unique key constraint	BE2	3
10.2	Middleware that hashes the request, stores and replays responses	BE1	10
10.3	Concurrency test with 50 parallel identical requests	QA	6
10.4	Key expiry (24 hours) and clean-up job	BE2	3
10.5	Update OpenAPI documentation	BE1	2

US-06 – Wallet top-up (5 points, 37 hours)

Task	Description	Role	Hours
06.1	Top-up API endpoint with amount validation	BE2	6
06.2	Orchestrate: create transaction, call mock gateway, update state, credit wallet	BE1	10
06.3	Wallet credit with row locking	BE2	5
06.4	Top-up screen showing live status	FE	8
06.5	Integration tests for success, decline,	QA	8

Task	Description	Role	Hours
	timeout and invalid amount		

Team tasks (36 hours)

Task	Description	Role	Hours
X.1	Docker Compose development and staging environment	DevOps	8
X.2	React application shell: routing, authentication context, API client	FE	8
X.3	Code review, pairing and Sprint 2 backlog refinement allowance	Team	16
X.4	Sprint review demo preparation	Team	4

5.3 Capacity and Commitment Check

Item	Value
Developers	5
Working days in sprint	10
Productive hours per person per day	6
Gross capacity	300 hours
Less Scrum events (planning, stand-ups, review, retrospective), about 4 hours per person	20 hours
Net capacity	280 hours
Planned work (35 story tasks, 168 hours, plus 4 team tasks, 36 hours)	204 hours
Utilisation	73%
Buffer for defects, impediments and first-sprint uncertainty	76 hours (27%)
Commitment	29 story points, about 7 hours per point

Planned hours by role: BE1 58, BE2 41, QA 38, FE 26, DevOps 21, shared team tasks 20. The load is uneven because Sprint 1 is backend-heavy; spare frontend and DevOps time is used for pairing, test automation, code review and fixing defects, which also spreads knowledge across the team. The

commitment is deliberately conservative because the team has no measured velocity yet; the actual velocity from this sprint will guide Sprint 2.

5.4 Burn-down Plan

Sprint day	0	2	4	6	8	10
Ideal story points remaining	29	23.2	17.4	11.6	5.8	0
Ideal task hours remaining	204	163	122	82	41	0

The team updates the burn-down chart at the daily stand-up. A flat line for two consecutive days triggers a discussion of impediments or scope.

6. Story Point Estimation

6.1 Technique

The team uses **Planning Poker with the Fibonacci sequence (1, 2, 3, 5, 8, 13)**. A story point is a relative measure of the total size of a story, combining **effort, complexity and uncertainty**; it is not a number of hours. The widening gaps in the Fibonacci scale reflect the fact that larger items are less certain to estimate.

Process: the Product Owner reads the story and answers questions; each developer privately chooses a card; all cards are revealed together; the highest and lowest voters explain their reasoning; the team votes again until the estimates converge. Any story estimated above 8 points is split before it can enter a sprint.

Reference story: US-04 (automatic wallet creation) is the baseline at **2 points**: one table, one trigger and simple tests, with no unknowns. Every other story is sized relative to it.

6.2 Sprint 1 Estimates and Justification

ID	Story	Points	Complexity	Uncertainty	Effort	Justification
US-04	Automatic wallet creation	2	Low	Low	Low	Reference story: one table, one transactional trigger, three simple test scenarios
US-26	Automated CI pipeline	3	Low	Low	Medium	Well-known tooling and templates, but several steps to

ID	Story	Points	Complexity	Uncertainty	Effort	Justification
						configure (build, lint, tests, scan, coverage gate)
US-01	Register and verify account	3	Low	Low	Medium	Standard create-and-verify flow with hashing and OTP via a mock provider, plus one screen
US-02	Secure login	3	Medium	Low	Medium	Token issuance, lockout and middleware are well-understood patterns; security tests add effort
US-12	Gateway interface and mock gateway	3	Medium	Low	Medium	Interface design plus a configurable mock; no external dependency; contract tests needed
US-08	Core transaction state machine	5	High	Medium	Medium	Core domain logic where correctness is critical and every transition must be tested;

ID	Story	Points	Complexity	Uncertainty	Effort	Justification
						scope limited to the top-up life-cycle
US-10	Idempotent requests	5	High	Medium	Medium	Concurrency-safe middleware, request hashing, response replay and expiry; parallel-request testing is non-trivial
US-06	Wallet top-up	5	Medium	Medium	High	Joins API, state machine, adapter, wallet credit with row locking and a screen; the most integration points and the largest task list (37 hours)

Why not larger: US-08, US-10 and US-06 each fit within one sprint and are far from the 13-point threshold at which the team would split a story. **Why not smaller:** each involves concurrency or several integration points, so they are clearly bigger than the 2-point reference and the 3-point stories.

6.3 Planning Poker Record (illustrative workshop round)

The votes below are a simulated record to show how the technique was applied in the workshop.

Story	Round 1 votes	Discussion	Final
US-08 State machine	3, 5, 5, 8, 13	The 13 assumed every state (reservation, reversal, unknown); the 3 assumed a simple status field. The team	5

Story	Round 1 votes	Discussion	Final
		agreed to model only the states needed for the top-up flow and defer the rest to US-07, US-16 and US-17	
US-10 Idempotency	3, 5, 5, 8, 8	The higher votes counted concurrency testing and response replay; the team agreed to keep key expiry as a small separate task	5
US-06 Wallet top-up	5, 5, 5, 8, 8	The 8 voters worried about UI work; after confirming the frontend shell is a separate team task (X.2), they accepted 5	5

6.4 Sanity Checks

- **Capacity check:** 29 points correspond to 204 planned hours, about 7 hours per point, using 73% of net capacity.
- **Relative check:** the three 5-point stories carry 24, 24 and 37 task hours and the 3-point stories carry 13 to 24 hours, so sizes are consistent with the task breakdown.
- **Velocity:** no history exists yet, so 29 points is a conservative first commitment. After Sprint 1 the measured velocity replaces this assumption. The whole backlog of 146 points is about five feature sprints plus a hardening sprint.

7. Sprint Goal and Expected Deliverables

7.1 Sprint Goal

By the end of Sprint 1, a verified user can register, log in and top up their wallet through a mock gateway, with every transaction tracked by a controlled state machine and protected from duplicate processing, all built and tested through an automated CI pipeline.

The goal is a single, testable sentence that gives the team a reason to make trade-offs: if time runs short, the team protects the top-up flow and its safeguards before any polish.

7.2 Expected Deliverables

#	Deliverable	What will be demonstrated at the sprint review	Stories

#	Deliverable	What will be demonstrated at the sprint review	Stories
D1	Automated CI pipeline	A pull request triggers build, tests and scan; a failing test blocks the merge	US-26
D2	Registration, verification and login	A new user registers, enters an OTP, logs in and receives a token; lockout after 5 wrong attempts	US-01, US-02
D3	Automatic wallet	A wallet with balance 0.00 exists as soon as the account is verified	US-04
D4	Gateway interface and mock gateway	The same call succeeds, is declined or times out depending on the mock mode	US-12
D5	Transaction state machine with history	A transaction timeline with timestamps; an illegal transition is rejected	US-08
D6	Idempotent API	The same top-up sent 50 times in parallel produces one transaction and one credit	US-10
D7	Wallet top-up end to end	A customer tops up through the screen, sees the live status and the new balance	US-06
D8	Supporting artifacts	OpenAPI documentation, state diagram, test and coverage report, Docker Compose environment, updated backlog and burn-down chart	All

Not promised in Sprint 1: payments to merchants, real gateways, OTP for payments, ledger, reconciliation and monitoring dashboards. These are planned for later sprints.

7.3 Definition of Done

A story is done only when all of the following are true:

1. Code is peer-reviewed and merged to the main branch through a pull request.
2. All acceptance criteria pass as automated tests; unit test coverage of the new code is at least 70%.
3. The CI pipeline is green, including the security and dependency scan.
4. No open high-severity defects remain on the story.
5. API changes are documented in OpenAPI and the state diagram is updated where relevant.
6. The feature is deployed to the staging environment and demonstrated to the Product Owner.
7. No sensitive data (passwords, tokens, OTPs) appears in logs.

7.4 Sprint 1 Events

Day	Event	Duration	Purpose
1	Sprint planning	4 hours	Confirm the sprint goal, selected stories, tasks and commitment
1–10	Daily stand-up	15 minutes	What was done, what is planned, what blocks progress
6	Backlog refinement	2 hours	Prepare and estimate Sprint 2 stories
10	Sprint review	1.5 hours	Demonstrate deliverables D1 to D8 to stakeholders and collect feedback
10	Sprint retrospective	1.5 hours	Agree concrete improvements for Sprint 2

7.5 Sprint 1 Risks

Risk	Mitigation
Scope creep in the state machine (adding reservation, reversal and unknown states early)	Sprint goal and the agreed top-up-only scope; extra states are separate backlog stories
Concurrency defects in idempotency and wallet credit	Early parallel-request tests; database unique constraints and row locking; pairing between BE1 and BE2
Team new to the pipeline and tools	Stories US-26 and X.1 are scheduled first;

Risk	Mitigation
	DevOps pairs with developers
Mock gateway behaves differently from a real provider	Contract tests keep the adapter interface strict; the real sandbox is integrated in Sprint 3

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25